Minimal QUBO for Telecom Backhaul Allocation

Goal. Allocate limited backhaul bandwidth to service streams while respecting SLAs. The problem is cast as a Quadratic Unconstrained Binary Optimisation (QUBO) so it can be solved directly on a D-Wave quantum annealer or any compatible sampler.

Minimal Case

Two streams $s \in \{1, 2\}$ share one bottleneck link with capacity C Mbps. The solver must decide, in discrete *chunk* steps of size Δ Mbps, how much bandwidth each stream receives.

Variable Glossary

Problem Parameters (inputs, not optimised)

Symbol	Meaning
C	Link capacity in Mbps. Hard upper bound on the total bandwidth that can be assigned across all streams sharing that link. *Code:* `Link.capacity_mbps`.
Δ	Chunk size in Mbps. The smallest unit of bandwidth that can be allocated. A finer Δ gives more resolution but adds more binary variables. *Code:* `delta_mbps` (default 10).
D_s	Demand of stream s in Mbps: the bandwidth it ideally wants. *Code:* `Stream.demand_mbps`.
G_s	Minimum guarantee of stream s in Mbps: the floor below which the allocation must not fall (SLA floor). *Code:* `Stream.min_guarantee_mbps`.
w_s	Priority weight of stream s. A higher value means shortfall for this stream is penalised more heavily. Voice streams typically carry $w_s \approx 3$–4; best-effort data carries $w_s \approx 1$. *Code:* `Stream.priority_weight`.
B_s^{prev}	Bandwidth allocated to stream s in the *previous* scheduling interval. Used to penalise large swings that would disrupt in-flight flows. *Code:* `Stream.prev_alloc_mbps`.
K_s	Number of allocation chunks available for stream s, $K_s = \lceil \max(D_s, G_s)/\Delta \rceil$. *Code:* `max_chunks_by_stream[s.name]`.
λ_{sf}	Penalty strength for the shortfall equality constraint. Governs how strongly the solver is pushed to satisfy $B_s + q_s = D_s$. *Code:* `lambda_shortfall` (default 6.0).
λ_{st}	Penalty strength for the stability term. Kept small relative to λ_{sf} so that stability is a soft preference, not a hard requirement. *Code:* `lambda_stability` (default 0.03).
λ_{cap}	Penalty strength for the link-capacity constraint. Should be the largest λ to make capacity violation the most expensive outcome. *Code:* `lambda_capacity` (default 15.0).
λ_{min}	Penalty strength for the minimum-guarantee constraint. Set between λ_{sf} and λ_{cap} to prioritise guarantees over full-demand satisfaction. *Code:* `lambda_min_guarantee` (default 12.0).
α	Linear cost coefficient applied to the shortfall. Together with w_s it provides a *direct* per-Mbps shortfall cost in addition to the quadratic penalty. *Code:* embedded as `sf_linear_weight = lambda_shortfall * s.priority_weight`.

Decision Variables (binary, optimised by the solver)

Symbol	Meaning
$y_{s,k} \in \{0,1\}$	Allocation bit for stream s, chunk index $k = 0, \ldots, K_s - 1$. Setting $y_{s,k} = 1$ grants Δ Mbps to stream s. *Code variable name:* `y::{stream}::{k}`.
B_s	Derived (not a QUBO variable itself): total bandwidth allocated to stream s, $$B_s = \Delta \sum_{k=0}^{K_s-1} y_{s,k}.$$ It is a linear function of the $y_{s,k}$ bits and appears inside every penalty term.
$q_{s,b} \in \{0,1\}$	Shortfall bit b for stream s. Together these bits binary-encode the integer shortfall in chunk units: $$q_s = \Delta \sum_b 2^b \, q_{s,b}.$$ q_s represents how many Mbps stream s falls short of its demand D_s. It is introduced so that the equality $B_s + q_s = D_s$ can be written as a quadratic penalty. *Code variable name:* `sf::{stream}::{b}`. Number of bits: $\lceil \log_2 K_s \rceil + 1$.
$u_b \in \{0,1\}$	Capacity slack bit b for the link. Together these bits binary-encode the unused link capacity: $$u = \Delta \sum_b 2^b \, u_b, \quad u \geq 0.$$ u lets the sum $B_1 + B_2 + u = C$ be turned into a penalty that is zero exactly when the constraint is tight or under-utilised. *Code variable name:* `csl::{link}::{b}`.
$r_{s,b} \in \{0,1\}$	Guarantee-slack bit b for stream s. Together these bits binary-encode the surplus above the guarantee floor: $$r_s = \Delta \sum_b 2^b \, r_{s,b}, \quad r_s \geq 0.$$ r_s captures how far above G_s the allocation sits. The equality $B_s - r_s = G_s$ is zero only when $B_s \geq G_s$, so any violation raises the energy. *Code variable name:* `gsl::{stream}::{b}`.

Why binary encode continuous slack? QUBO requires every variable to be binary ($\{0,1\}$). Non-negative integers like q_s are represented in binary via the standard basis $q_s = \sum_b 2^b q_{s,b}$, scaled by Δ to stay in Mbps units. The number of bits needed is $\lceil \log_2(\text{max value}) \rceil + 1$, computed in the code by `bit_width_for_nonnegative_integer`.

Objective Terms

We minimise a weighted sum of penalty terms. Each term is zero when the corresponding constraint or preference is exactly satisfied, and grows quadratically with the violation.

1) Shortfall (soft demand satisfaction)

We want B_s to be as close to D_s as possible. Introduce the shortfall $q_s \geq 0$ and enforce the equality:

$$B_s + q_s = D_s, \quad q_s \geq 0.$$

The quadratic penalty drives the equality to zero; the linear term gives a per-Mbps cost proportional to the stream's priority weight:

$$\lambda_{\text{sf}} \sum_{s=1}^{2} (B_s + q_s - D_s)^2 + \alpha \sum_{s=1}^{2} w_s \, q_s.$$

When allocation equals demand ($B_s = D_s$) the optimal $q_s = 0$ and both terms vanish. When the link is over-subscribed, the solver raises q_s for lower-priority streams first because their w_s is smaller, incurring a lower linear cost.

2) Stability (allocation smoothness)

Large shifts between intervals disrupt TCP flows and increase re-ordering. We penalise the squared distance from the previous allocation:

$$\lambda_{\text{st}} \sum_{s=1}^{2} (B_s - B_s^{\text{prev}})^2.$$

$\lambda_{\text{st}} \ll \lambda_{\text{sf}}$ so this is a soft preference: the solver will deviate from B_s^{prev} when necessary to satisfy demand or capacity, but avoids unnecessary churn.

Constraints as Quadratic Penalties

Hard constraints cannot be expressed in QUBO directly; instead they are added as penalty terms whose coefficients are large enough that any constraint violation costs more than any objective gain.

3) Link capacity

The total bandwidth on the link must not exceed C. Introduce nonnegative slack u representing unused capacity:

$$B_1 + B_2 + u = C, \quad u \geq 0,$$

$$\lambda_{\text{cap}} (B_1 + B_2 + u - C)^2.$$

u absorbs the gap between total allocation and the capacity ceiling. When the link is exactly full, $u = 0$. When under-utilised, $u = C - B_1 - B_2 > 0$. Over-subscription ($B_1 + B_2 > C$) cannot be absorbed by u (it is non-negative), so the penalty is non-zero, repelling the solver from that region. λ_{cap} is the largest penalty coefficient to make capacity violation the most expensive outcome in the energy landscape.

4) Minimum guarantees

Each stream must receive at least G_s Mbps. A nonnegative slack r_s captures the surplus:

$$B_s - r_s = G_s, \quad r_s \geq 0,$$

$$\lambda_{\min} \sum_{s=1}^{2} (B_s - r_s - G_s)^2.$$

When $B_s \geq G_s$ the solver can set $r_s = B_s - G_s \geq 0$ and the penalty vanishes. When $B_s < G_s$ there is no valid non-negative r_s; the penalty is strictly positive, steering the solver away from under-provisioning.

Final QUBO Energy

Combining all terms gives the energy function that the annealer minimises:

$$E(\mathbf{x}) = \underbrace{\lambda_{\text{sf}} \sum_{s=1}^{2} (B_s + q_s - D_s)^2 + \alpha \sum_{s=1}^{2} w_s q_s}_{\text{shortfall cost}}$$

$$+ \underbrace{\lambda_{\text{st}} \sum_{s=1}^{2} (B_s - B_s^{\text{prev}})^2}_{\text{stability cost}} + \underbrace{\lambda_{\text{cap}} (B_1 + B_2 + u - C)^2}_{\text{capacity constraint}} + \underbrace{\lambda_{\text{min}} \sum_{s=1}^{2} (B_s - r_s - G_s)^2}_{\text{guarantee constraint}} .$$

All variables are binary after encoding q_s, u, r_s with bit vectors. The expansion of each squared term produces the linear (diagonal) and quadratic (off-diagonal) QUBO coefficients added by `add_square_penalty` in the code.

Penalty coefficient ordering. For constraints to dominate objectives the coefficients must satisfy

$$\lambda_{\text{cap}} > \lambda_{\text{min}} > \lambda_{\text{sf}} \gg \lambda_{\text{st}}.$$

The defaults (15, 12, 6, 0.03) reflect this hierarchy: capacity is never violated, guarantees are protected next, demand satisfaction comes third, and stability is a soft tie-breaker.

Tiny Numeric Example

Let $\Delta = 10$ Mbps, $C = 100$, $D_1 = 70$, $D_2 = 60$, $G_1 = 40$, $G_2 = 30$. Then $D_1 + D_2 = 130 > C$, so the optimizer must throttle.

Stream 1 and 2 each get $K_s = \lceil D_s/\Delta \rceil = 7$ and 6 allocation bits respectively, plus shortfall and slack bits, for a total variable count well within classical sampler reach.

If $w_1 > w_2$, the shortfall linear term penalises stream 2's shortfall less than stream 1's. The optimal QUBO solution therefore keeps B_1 closer to $D_1 = 70$ and assigns the larger shortfall to stream 2, while still respecting both minimum guarantees ($G_1 = 40$, $G_2 = 30$) via the capacity-slack variable u that absorbs the remaining $100 - B_1 - B_2$ Mbps.